

# Лекция №9 Автоматизация пользовательского интерфейса

Для автоматического тестирования приложений Windows Forms существует несколько подходов и инструментов:

## Популярные инструменты для тестирования

1. **UI Automation (UIA) Framework** - встроенный в Windows фреймворк для автоматизации
2. **WinAppDriver** - Microsoft инструмент для автоматизации Windows приложений
3. **TestStack.White** - библиотека для автоматизации UI (устаревшая)
4. **FlaUI** - современная замена White, работает с UIA2 и UIA3
5. **Appium** - кроссплатформенный инструмент (может работать с WinAppDriver)

## 1. WinAppDriver (Windows Application Driver)

**Microsoft** предоставляет официальный инструмент для автоматизации тестирования Windows-приложений, включая WinForms и WPF.

### Как начать:

1. **Установка:**
  - Скачайте и установите [WinAppDriver](#).
  - Запустите сервер (WinAppDriver.exe).
2. **Пример теста на C# (с использованием Selenium WebDriver + WinAppDriver):**

```
using OpenQA.Selenium.Appium.Windows;  
using OpenQA.Selenium.Remote;  
using NUnit.Framework;
```

```
[TestFixture]  
public class WinFormsTest  
{  
    private WindowsDriver<WindowsElement> _driver;
```

```

[SetUp]
public void Setup()
{
    var appCapabilities = new DesiredCapabilities();
    appCapabilities.SetCapability("app", @"C:\Path\To\YourApp.exe");
    appCapabilities.SetCapability("deviceName", "WindowsPC");
    _driver = new WindowsDriver<WindowsElement>(new
Uri("http://127.0.0.1:4723"), appCapabilities);
}

[Test]
public void TestButtonClick()
{
    var button = _driver.FindElementByAccessibilityId("btnSubmit");
    button.Click();
    Assert.IsTrue(_driver.FindElementByAccessibilityId("lblStatus").Text ==
"Success");
}

[TearDown]
public void TearDown() => _driver?.Quit();
}

```

#### Примечание:

- Используйте **AccessibilityId** (аналог AutomationId в WinForms) для поиска элементов.
- WinAppDriver поддерживает **XPath**, **Name**, **ClassName** и другие локаторы.

## 2. FlaUI (альтернатива WinAppDriver)

**FlaUI** — это современная .NET-библиотека для UI-автоматизации, работающая поверх **UIA2** (MSAA) и **UIA3** (UI Automation).

#### Преимущества:

- ✓ Не требует запущенного сервера (в отличие от WinAppDriver).
- ✓ Поддерживает **WinForms**, **WPF**, **UWP**.
- ✓ Работает быстрее, чем WinAppDriver.

#### Пример теста:

```

using FlaUI.UIA3;
using FlaUI.Core.AutomationElements;
using NUnit.Framework;

[TestFixture]
public class FlaUITest
{
    [Test]
    public void TestLoginForm()
    {
        // Запуск приложения
        var app = FlaUI.Core.Application.Launch(@"C:\Path\To\YourApp.exe");

        using (var automation = new UIA3Automation())
        {
            var window = app.GetMainWindow(automation);

            // Находим элементы по AutomationId
            var loginInput = window.FindFirstDescendant(cf =>
cf.ByAutomationId("txtUsername"))?.AsTextBox();
            var passwordInput = window.FindFirstDescendant(cf =>
cf.ByAutomationId("txtPassword"))?.AsTextBox();
            var loginButton = window.FindFirstDescendant(cf =>
cf.ByAutomationId("btnLogin"))?.AsButton();

            // Взаимодействие
            loginInput?.Enter("admin");
            passwordInput?.Enter("12345");
            loginButton?.Click();

            // Проверка результата
            var statusLabel = window.FindFirstDescendant(cf =>
cf.ByAutomationId("lblStatus"))?.AsLabel();
            Assert.AreEqual("Login successful", statusLabel?.Text);
        }

        app.Close();
    }
}

```

---

### 3. Интеграция с NUnit/xUnit (Unit-тесты для UI)

Можно комбинировать **FlaUI** или **WinAppDriver** с тестовыми фреймворками (**NUnit**, **xUnit**, **MSTest**).

### Пример с NUnit:

```
[TestFixture]
public class LoginTests
{
    private WindowsDriver<WindowsElement> _driver;

    [OneTimeSetUp]
    public void StartApp()
    {
        var options = new AppiumOptions();
        options.AddAdditionalCapability("app", @"C:\YourApp.exe");
        options.AddAdditionalCapability("deviceName", "WindowsPC");
        _driver = new WindowsDriver<WindowsElement>(new Uri("http://127.0.0.1:4723"),
options);
    }

    [Test]
    public void Login_ValidCredentials_Success()
    {
        _driver.FindElementByAccessibilityId("txtUsername").SendKeys("admin");
        _driver.FindElementByAccessibilityId("txtPassword").SendKeys("12345");
        _driver.FindElementByAccessibilityId("btnLogin").Click();

        var message = _driver.FindElementByAccessibilityId("lblStatus").Text;
        Assert.AreEqual("Welcome, admin!", message);
    }

    [OneTimeTearDown]
    public void CloseApp() => _driver?.Quit();
}
```

---

## Вывод: что выбрать?

| Инструмент   | Язык | Требует сервер? | Скорость | Поддержка WinForms |
|--------------|------|-----------------|----------|--------------------|
| WinAppDriver | C#   | Да              | Средняя  | ✓                  |
| FlaUI        | C#   | Нет             | Высокая  | ✓                  |

- Для .NET-разработчиков → FlaUI (быстрее, проще в настройке).
- Для интеграции с Selenium → WinAppDriver.

UI Automation (UIA) Framework – это встроенный в Windows фреймворк, предназначенный для автоматизации тестирования и взаимодействия с пользовательским интерфейсом (UI) приложений. Он предоставляет программируемый доступ к элементам интерфейса, позволяя автоматизировать действия пользователя (клики, ввод текста, навигацию и т. д.) и получать информацию о состоянии элементов.

### ◆ Основные возможности UIA

#### 1. Автоматизация любых UI-элементов

- Работает с Win32, WPF, WinForms, UWP, браузерами (через MSAA/UIA-поддержку) и даже некоторыми Java/Swing-приложениями.
- Поддерживает современные технологии, такие как DirectUI (используется в Windows 10/11).

#### 2. Доступ к дереву элементов

- UIA представляет интерфейс приложения в виде иерархического дерева (UIA Tree), аналогичного DOM в вебе.
- Можно находить элементы по:
  - AutomationId (уникальный идентификатор)
  - Name (текст элемента)

- `ControlType` (тип элемента: кнопка, поле ввода и т. д.)
- `Class Name` (имя класса в Win32)

### 3. Чтение и изменение свойств элементов

- Получение текста (`ValuePattern` для текстовых полей).
- Проверка состояния (`TogglePattern` для чекбоксов).
- Навигация (`TreeWalker` для перемещения по дереву элементов).

### 4. Взаимодействие с элементами

- Клики (`InvokePattern` для кнопок).
- Ввод текста (`ValuePattern` для полей ввода).
- Выбор элементов (`SelectionPattern` для списков).

### 5. Поддержка событий

- Можно подписаться на события: изменение текста, фокусировка, клик и т. д.

---

## ♦ Как работает UIA?

UIA использует **Microsoft Active Accessibility (MSAA)** как основу, но предоставляет более мощный и гибкий API.

### Основные компоненты UIA:

1. **UIAutomation** – главный интерфейс для работы с UIA.
2. **AutomationElement** – представляет UI-элемент (кнопку, поле ввода и т. д.).
3. **Control Patterns** – шаблоны взаимодействия (например, `InvokePattern` для клика).
4. **TreeWalker** – навигация по дереву элементов.
5. **Condition** – условия поиска элементов (`PropertyCondition`, `AndCondition`).

## ♦ Примеры использования

### 1. Поиск кнопки по имени и клик (C#)

```
using System.Windows.Automation;
```

```
var root = AutomationElement.RootElement; // Корневой элемент (рабочий стол)
```

```
var condition = new PropertyCondition(AutomationElement.NameProperty,  
"Открыть");
```

```
var button = root.FindFirst(TreeScope.Descendants, condition);

var invokePattern = button.GetCurrentPattern(InvokePattern.Pattern) as
InvokePattern;
invokePattern.Invoke(); // Клик по кнопке
```

## 2. Получение текста из поля ввода

```
var condition = new PropertyCondition(AutomationElement.AutomationIdProperty,
"txtUsername");

var textBox = root.FindFirst(TreeScope.Descendants, condition);

var valuePattern = textBox.GetCurrentPattern(ValuePattern.Pattern) as ValuePattern;
Console.WriteLine(valuePattern.Current.Value); // Выводит текст из поля
```

## 3. Подписка на событие изменения текста

```
var changedEvent = Automation.AddAutomationPropertyChangedEvent(

    textBox,

    TreeScope.Element,

    (sender, e) => Console.WriteLine("Текст изменился!"),

    ValuePattern.ValueProperty
);
```

# Инструменты для работы с UIA

1. **Inspect.exe** (из Windows SDK) – просмотр дерева элементов и их свойств. Чтобы узнать `AutomationId` элементов. Но мы будем использовать **FlaUIInspect**. скачать <https://github.com/FlaUI/FlaUIInspect>
2. **FlaUI** (библиотека для C#) – упрощённая обёртка над UIA.

Для работы с **UI Automation (UIA)** + **FlaUI** в проекте нужно добавить **специальный тестовый проект** (например, консольное приложение или NUnit-проект).

- ♦ **Что нужно добавить в решение?**

## 1. Новый проект (тип: Console App или NUnit Test Project)

- Это будет отдельный проект для автоматизированного тестирования.
- Пример структуры решения:

Solution 'ProductManager'

├── ProductManager (WinForms-приложение)  
└── ProductManager.UITests (тесты на FlaUI)

## Установка NuGet-пакетов

В тестовый проект нужно добавить:

- FlaUI.UIA3
- MSTest.TestAdapter
- MSTest.TestFramework

Ниже приведены необходимые директивы для работы с FlaUI.

```
using FlaUI.UIA3;  
using FlaUI.Core;  
using FlaUI.Core.AutomationElements;  
using Microsoft.VisualStudio.TestTools.UnitTesting;  
using FlaUI.UIA3.Patterns;  
using FlaUI.Core.Definitions;
```

## ◆ Важно!

### 1. AutomationId в WinForms

- Чтобы FlaUI находил элементы, задайте Name в свойствах контролов (**это будет их AutomationId**):

```
this.nameTextBox.Name = "nameTextBox";  
this.button1.Name = "button1";
```

- Или используйте поиск по другим свойствам (например, `ByText("Добавить")`).

### 2. Запуск приложения из тестов

- Укажите правильный путь к приложению `Application.exe`:
- `var app = Application.Launch(@"bin\Debug\Application.exe");`



## ◆ Основные способы задания условий поиска (Condition)

В FlaUI условия задаются через `ConditionFactory` (доступен через `cf` в примерах).

### 1. Поиск по AutomationId

```
var element = window.FindFirstDescendant(cf => cf.ByAutomationId("nameTextBox"));
```

- Идеально для WinForms/WPF, если у элементов задан Name (в WinForms Name становится AutomationId).

### 2. Поиск по тексту (NameProperty)

```
var element = window.FindFirstDescendant(cf => cf.ByName("Добавить"));
```

- Работает для кнопок, лейблов и других элементов с видимым текстом.

### 3. Поиск по типу элемента (ControlType)

```
var button = window.FindFirstDescendant(cf => cf.ByControlType(FlaUI.Core.Definitions.ControlType.Button));
```

- Полезно, если у элемента нет AutomationId.

### 4. Поиск по классу (ClassName)

```
var element = window.FindFirstDescendant(cf => cf.ByClassName("WindowsForms10.EDIT"));
```

- Актуально для Win32/WinForms (классы окон, например, Button, Edit).

### 5. Комбинированные условия (And/Or)

```
var element = window.FindFirstDescendant(cf => cf.ByControlType(FlaUI.Core.Definitions.ControlType.Button)
    .And(cf.ByName("Удалить"))
);
```

---

## ◆ Дополнительные методы поиска

### 1. FindAllDescendants – поиск всех подходящих элементов

```
var buttons = window.FindAllDescendants(cf => cf.ByControlType(FlaUI.Core.Definitions.ControlType.Button));
```

```
foreach (var button in buttons)
{
    Console.WriteLine(button.Name);
}
```

## 2. Поиск по индексу (если несколько одинаковых элементов)

```
var thirdButton = window.FindAllDescendants(cf =>
cf.ByControlType(FlaUI.Core.Definitions.ControlType.Button))[2];
```

## 3. Поиск вложенных элементов (XPath-подобный)

```
var listItem = window.FindFirstDescendant(cf => cf.ByAutomationId("productsList"))
    .FindFirstDescendant(cf =>
cf.ByControlType(FlaUI.Core.Definitions.ControlType.ListItem));
```

---

# ◆ Примеры для приложения ProductManager.exe

## 1. Поиск кнопки "Добавить"

```
var addButton = mainWindow.FindFirstDescendant(cf =>
    cf.ByAutomationId("button1") // Если задан Name
    // Или:
    // cf.ByName("Добавить")
    //
    cf.ByControlType(FlaUI.Core.Definitions.ControlType.Button).And(cf.ByName("Добавит
ь")))
);
```

## 2. Поиск поля ввода цены

```
var priceInput = mainWindow.FindFirstDescendant(cf =>
    cf.ByAutomationId("numericUpDown1") // Для NumericUpDown
);
```

## 3. Поиск списка товаров

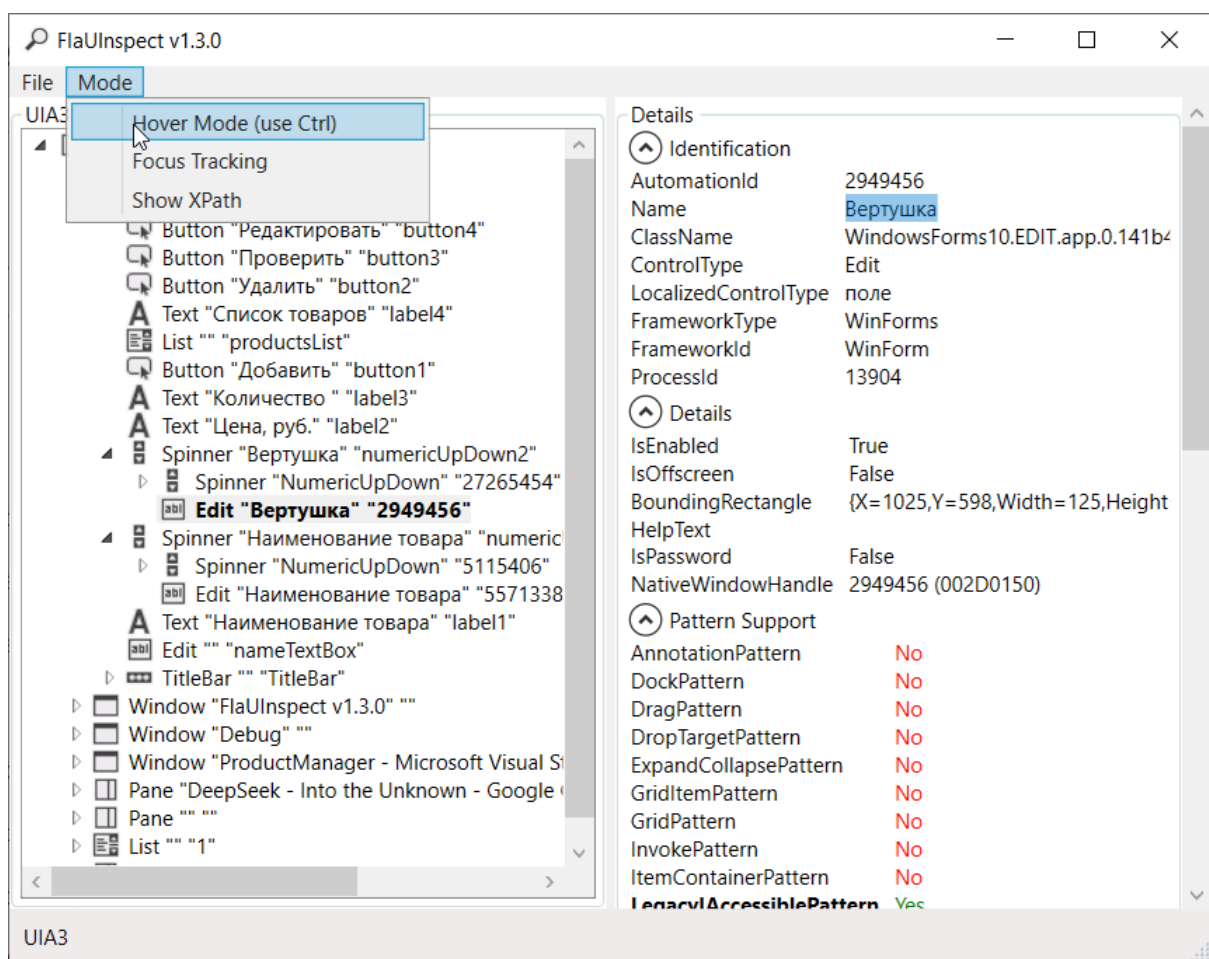
```
var productsList = mainWindow.FindFirstDescendant(cf =>
    cf.ByAutomationId("productsList") // ListBox в WinForms
);
```

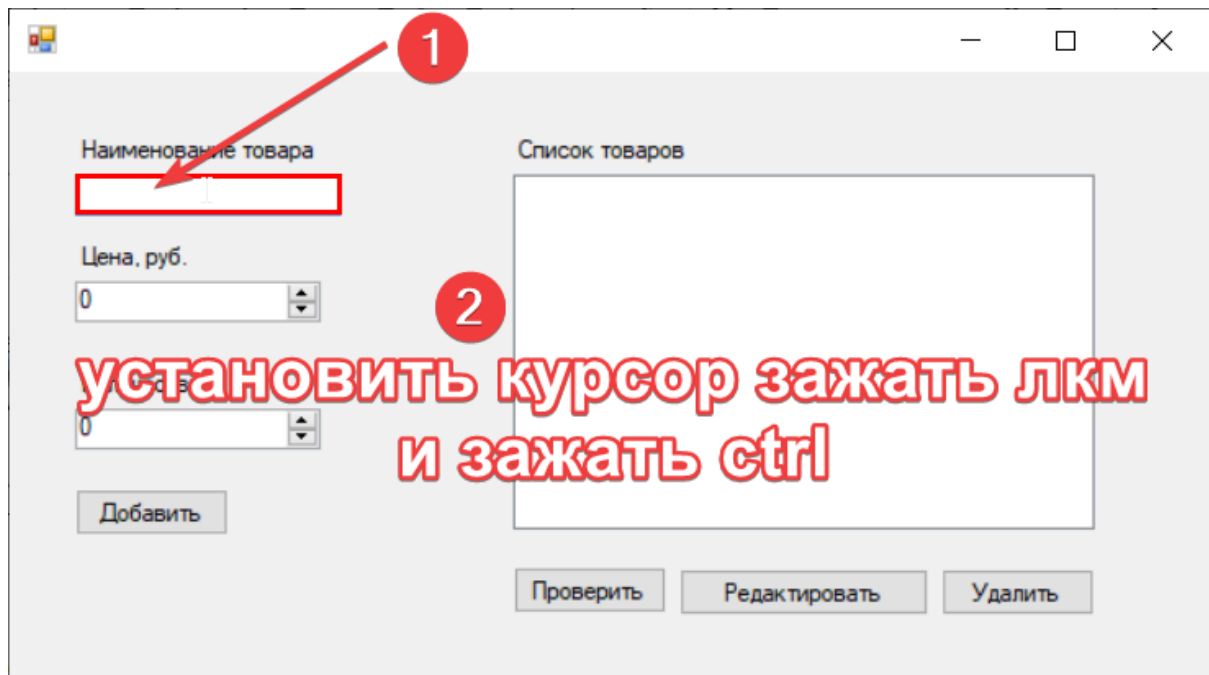
---

## ♦ Как узнать свойства элемента?

Используйте **FlaUIInspect**:

- Запустите приложение.
- Наведите курсор и зажмите ctrl на элемент и посмотрите:
  - AutomationId
  - Name
  - ControlType
  - ClassName





## ♦ Итог

| Метод/Условие           | Пример использования                                                        | Когда применять?                      |
|-------------------------|-----------------------------------------------------------------------------|---------------------------------------|
| ByAutomationId          | <code>cf.ByAutomationId("nameTextBox")</code>                               | Если у элемента есть Name (WinForms). |
| ByName / ByText         | <code>cf.ByName("Добавить")</code>                                          | Для кнопок, лейблов с текстом.        |
| ByControlType           | <code>cf.ByControlType(ControlType.Button)</code>                           | Если нет других идентификаторов.      |
| ByClassName             | <code>cf.ByClassName("WindowsForms10.BUTTON")</code>                        | Для Win32/WinForms-элементов.         |
| Комбинированные условия | <code>cf.ByName("Удалить").And(cf.ByControlType(ControlType.Button))</code> | Для точного поиска.                   |

**Совет:** Всегда старайтесь использовать **AutomationId** — это самый надежный способ.

После того как элемент найден с помощью `FindFirstDescendant` или других методов поиска в **FlaUI/UIA**, у него можно вызывать различные методы и свойства для взаимодействия и проверки состояния.

◆ **Основные методы и свойства элементов**

**1. Общие методы (для всех элементов)**

| Метод/Свойство                | Описание                                                       | Пример                                                              |
|-------------------------------|----------------------------------------------------------------|---------------------------------------------------------------------|
| <code>Click()</code>          | Клик по элементу (если поддерживается).                        | <code>button.Click()</code>                                         |
| <code>DoubleClick()</code>    | Двойной клик.                                                  | <code>listItem.DoubleClick()</code>                                 |
| <code>Focus()</code>          | Установить фокус на элемент.                                   | <code>textBox.Focus()</code>                                        |
| <code>SetValue(string)</code> | Установить значение (для текстовых полей).                     | <code>textBox.Text = "Новый текст"</code>                           |
| <code>GetValue()</code>       | Получить значение (например, текст).                           | <code>var text = label.Text</code>                                  |
| <code>IsEnabled</code>        | Проверить, доступен ли элемент.                                | <code>if (button.IsEnabled) {<br/>... }</code>                      |
| <code>IsOffscreen</code>      | Проверить, видим ли элемент на экране.                         | <code>if (!element.IsOffscreen)<br/>{ ... }</code>                  |
| <code>BoundingBox</code>      | Получить координаты элемента (x, y, ширина, высота).           | <code>var rect =<br/>element.BoundingBox</code>                     |
| <code>Name</code>             | Получить имя элемента (текст метки, заголовок кнопки и т. д.). | <code>var name = button.Name</code>                                 |
| <code>Patterns</code>         | Доступ к шаблонам элементов (см. ниже).                        | <code>var pattern =<br/>element.Patterns.Toggle.P<br/>attern</code> |

## 2. Методы для конкретных типов элементов

### ♦ TextBox (поле ввода)

| Метод/Свойство             | Описание                   | Пример                                    |
|----------------------------|----------------------------|-------------------------------------------|
| <code>Enter(string)</code> | Ввести текст.              | <code>textBox.Enter("Привет")</code>      |
| <code>Text</code>          | Получить/установить текст. | <code>textBox.Text = "Новый текст"</code> |

### ♦ Button (кнопка)

| Метод                 | Описание                                    | Пример                       |
|-----------------------|---------------------------------------------|------------------------------|
| <code>Invoke()</code> | Нажать кнопку (аналог <code>Click</code> ). | <code>button.Invoke()</code> |

### ♦ CheckBox/ToggleButton (чекбокс, переключатель)

| Метод                  | Описание                   | Пример                                     |
|------------------------|----------------------------|--------------------------------------------|
| <code>Toggle()</code>  | Переключить состояние.     | <code>checkBox.Toggle()</code>             |
| <code>IsChecked</code> | Получить состояние (bool). | <pre>if (checkBox.IsChecked) { ... }</pre> |

### ♦ ComboBox (выпадающий список)

| Метод                       | Описание                    | Пример                                      |
|-----------------------------|-----------------------------|---------------------------------------------|
| <code>Expand()</code>       | Раскрыть список.            | <code>comboBox.Expand()</code>              |
| <code>Collapse()</code>     | Свернуть список.            | <code>comboBox.Collapse()</code>            |
| <code>Select(индекс)</code> | Выбрать элемент по индексу. | <code>comboBox.Select(0)</code>             |
| <code>SelectedItem</code>   | Получить выбранный элемент. | <pre>var item = comboBox.SelectedItem</pre> |

♦ **ListBox (список элементов)**

| Метод                 | Описание                      | Пример                                                   |
|-----------------------|-------------------------------|----------------------------------------------------------|
| <b>Select(индекс)</b> | Выбрать элемент.              | <code>listBox.Select(1)</code>                           |
| <b>Items</b>          | Получить все элементы списка. | <code>foreach (var item in listBox.Items) { ... }</code> |

♦ **Slider (ползунок)**

| Метод                   | Описание                   | Пример                              |
|-------------------------|----------------------------|-------------------------------------|
| <b>SetValue(double)</b> | Установить значение.       | <code>slider.SetValue(50.0)</code>  |
| <b>Value</b>            | Получить текущее значение. | <code>var val = slider.Value</code> |

♦ **Работа с шаблонами элементов (UIA Patterns)**

Некоторые элементы поддерживают **специальные шаблоны** (Patterns), которые дают дополнительные методы.

**Основные шаблоны:**

| Шаблон                  | Описание                                   | Пример                                                        |
|-------------------------|--------------------------------------------|---------------------------------------------------------------|
| <b>InvokePattern</b>    | Для кнопок ( <code>Invoke()</code> ).      | <code>button.Patterns.Invoke.Pattern.Invoke()</code>          |
| <b>ValuePattern</b>     | Для полей ввода ( <code>SetValue</code> ). | <code>textBox.Patterns.Value.Pattern.SetValue("Текст")</code> |
| <b>TogglePattern</b>    | Для чекбоксов ( <code>Toggle</code> ).     | <code>checkBox.Patterns.Toggle.Pattern.Toggle()</code>        |
| <b>SelectionPattern</b> | Для списков ( <code>Select</code> ).       | <code>listBox.Patterns.Selection.Pattern.Select(0)</code>     |

**Пример:**

```
var togglePattern = checkBox.Patterns.Toggle.Pattern;
if (togglePattern.ToggleState == ToggleState.On)
{
    togglePattern.Toggle(); // Выключить чекбокс
}
```

## ◆ Примеры для приложения ProductManager

### 1. Добавление товара

```
var nameTextBox = mainWindow.FindFirstDescendant(cf =>
cf.ByAutomationId("nameTextBox")).AsTextBox();
var priceInput = mainWindow.FindFirstDescendant(cf =>
cf.ByAutomationId("numericUpDown1")).AsNumericUpDown();
var addButton = mainWindow.FindFirstDescendant(cf =>
cf.ByAutomationId("button1")).AsButton();
```

```
nameTextBox.Text = "Телевизор";
priceInput.Value = 25000;
addButton.Click(); // Или addButton.Invoke()
```

### 2. Проверка состояния кнопки

```
var deleteButton = mainWindow.FindFirstDescendant(cf =>
cf.ByAutomationId("button2")).AsButton();
if (deleteButton.IsEnabled)
{
    deleteButton.Click();
}
```

### 3. Получение списка товаров

```
var productsList = mainWindow.FindFirstDescendant(cf =>
cf.ByAutomationId("productsList")).AsListBox();
foreach (var item in productsList.Items)
{
    Console.WriteLine(item.Text); // Выводит "Название | Цена: X руб. | Количество: Y"
}
```

## ◆ Итог



| Тип элемента    | Основные методы                                   | Пример использования                   |
|-----------------|---------------------------------------------------|----------------------------------------|
| <b>TextBox</b>  | <code>Enter()</code> , <code>Text</code>          | <code>textBox.Enter("Текст")</code>    |
| <b>Button</b>   | <code>Click()</code> , <code>Invoke()</code>      | <code>button.Click()</code>            |
| <b>CheckBox</b> | <code>Toggle()</code> ,<br><code>IsChecked</code> | <code>checkBox.Toggle()</code>         |
| <b>ListBox</b>  | <code>Select()</code> , <code>Items</code>        | <code>listBox.Items[0].Select()</code> |
| <b>Slider</b>   | <code>SetValue()</code> , <code>Value</code>      | <code>slider.SetValue(30)</code>       |

**Совет:** Если элемент не реагирует на стандартные методы, проверьте его **Patterns** (например, `InvokePattern` для кнопок).